

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf\_471a9cdd-d1d\agent-a8d99acb23745e8d2.jsonl`
- SHA-256: `f6e738b8f624982cd7bb1ffce067601934eaf6ec9a8ef03ad2daf275c0ff1f22`
- Source modified: `2026-06-10T09:32:55+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf\_471a9cdd-d1d`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T09:28:10.500Z)

CONTEXT ? you are advising a real architecture decision for a solo-founder project (owner = avidullu).

Repos on disk (read-only; cite file paths for every claim):

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? FastAPI+SQLite+ffmpeg rally/highlight pipeline. Pluggable registries (segmenters/providers/validators/sports/annotations). Key seams: backend/pipeline/segmenters/base.py (registry), backend/pipeline/detectors/base.py (DetectorRunner ABC ? WASB/TrackNet run as WSL subprocesses from an EXTERNAL repo ~/models/WASB-SBDT), backend/providers/ (Gemini = external API). Eval stack backend/eval/ (metrics.match_intervals = the scoring spine, rally_seq_proto.py = the learned Gen-0 prototype seed, eval_partial_labels, calibrate_local LOVO, regression gate w/ tracked baseline in eval_baselines/). CI = .github/workflows/ci.yml (an import-smoke job WITHOUT the GPU stack + a full-suite job with CPU torch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? separate repo, the data system-of-record (golden labels, licensing gate, CVAT ingestion, 'curate export' manifest consumed by the indexer's batch eval).
- Also exists: private sports-obsreport repo = a shared report library consumed as a SOFT dependency by both repos (a working precedent for 'separate repo + consumed as dep').

THE DECISION: the owner is about to greenlight M0.4 (productionize rally_seq_proto into a Gen-0 TAL train?eval?ship-gate harness over ActionFormer/ASFormer (MIT), later Gen-2 = Qwen3-VL fine-tuned via ms-swift) and M0.1 (corpus spine + event-index contract, authored in the collector). QUESTION: should the model/training work live IN the indexer repo, or in a SEPARATE repo whose output is consumed by the indexer as just another segmenter/runner (like WASB and Gemini)? The owner PREFERS isolation ("iterate independently", cites the collector split as precedent) but asked for honest thoughts.

KNOWN FACTS to weigh (verified in a deep audit this week ? do not re-litigate, build on them):

- Cross-repo contract drift IS a real observed cost here: the collector's export header (start,end) vs the indexer's calibrate_wasb loader (start_time,end_time) forked into two incompatible golden-CSV conventions; tuned constants (TUNED/BASELINE) were duplicated-by-copy across files and drifted.
- Import-graph hygiene is a live concern: importing backend.main pulls torch/cv2 via the segmenter registration manifest (lazy-imports were deferred to the async-job slice).
- Authoritative plan docs: docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md ('envision now, build later' ? design every deferred seam up front; train EXPORT-CLEAN ops; on-device later = exported ONNX/CoreML/GGUF artifact + thin shell; the moat = consented dataset + eval harness, NOT the weights), docs/NEXT_STEPS.md, docs/PLATFORM_ARCHITECTURE.md.
- Guardrails: MIT/Apache-2.0/BSD only for code+data+weights deps; trained weights themselves = private/proprietary asset; minors excluded from training; per-user training opt-in; consent ledger planned in M0.1 schema.
- Quality state: learned LOVO mean 0.626 (n=6) vs heuristic 0.480; ship-gate F1@tIoU target still undefined.

- A repo RENAME off 'badminton' is already an owner action item ? repo topology is in flux anyway.
- Training runs on the owner's Windows box via WSL2 + RTX 2070 Super (8GB); Gen-2 may use rented cloud GPUs (Modal/RunPod per the study).
- Solo developer + AI agents do the work; there are ALREADY 4-5 repos (indexer, collector, rally-annotator, sports-obsreport).

RULES: read-only. Your final structured output is consumed programmatically.

ROLE: ML-infrastructure practitioner. Independent of repo topology, specify the train/serve boundary that MUST hold, then say which topology better enforces it for a solo dev + AI agents.

Read: backend/eval/rally_seq_proto.py (features computed from trajectories ? note the train-time featurizer must equal serve-time exactly; the fps-normalization bug just fixed there is a textbook train/serve-skew instance), backend/pipeline/detectors/base.py, docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md. You may WebSearch for prevailing practice (model registries, artifact manifests, monorepo-vs-polyrepo for ML, HF private hubs vs git-lfs vs releases). Deliver in boundary_design: the exact ARTIFACT CONTRACT (weights + featurizer + manifest fields: feature schema/version, normalization params, calibration/threshold, training-data lineage + consent fingerprint, eval fingerprint vs the n=6 corpus, license attestation), who owns the SHIP-GATE (recommend: the judge lives with the product?), and where feature code lives so train==serve. Then your topology recommendation.

2. user (2026-06-10T09:28:18.805Z)

```

1      """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
2      heuristic).
3      Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
4      rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
5      (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
6      the trajectory; the hand-tuned heuristic just can't extract it across footage. This
7      prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
8      recovers the rally segments ? including on the footage the heuristic fails on.
9
10     Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
11     LogisticRegression ? no torch/sklearn):
12         1. Per-frame features over a local window from the WASB trajectory: visibility rate,
13            mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
14            All are scale/camera-robust RATES (unlike a raw velocity threshold).
15         2. Per-frame label = inside a human GT rally or not.
16         3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
17            held-out one (the honest cross-footage number; no within-video leakage).
18         4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
19            -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
20            heuristic).
21
22     This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
23     hand features). It exists to answer one question: is the rally boundary learnable from
24     the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.
25
26     Run:
27     python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
28
29     from __future__ import annotations
30
31     import json
32     import argparse
33     from typing import Dict, List, Tuple
34
35     import numpy as np
36     from scipy.ndimage import uniform_filter1d, maximum_filter1d
37     from scipy.stats import rankdata
38
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
40     from backend.eval.classifier import LogisticRegression
41     from backend.eval.calibrate_wasb import load_golden_gts
42     from backend.eval.metrics import score

```

```

42
43 FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
44 Interval = Tuple[float, float]
45
46
47 def _spread(a: np.ndarray) -> float:
48     return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
49
50
51 def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
52     """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.
53
54     Speed is normalized to fraction-of-frame-width PER SECOND (`* fps`), matching the
55     heuristic windowing brain ? previously it was per-frame, so the same physical
shuttle
56     speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
transfer
57     on a corpus that mixes frame rates. Speed/crossings are also not computed across a
long
58     occlusion gap (mirrors the heuristic resetting on invisibility)."""
59     pts = sorted(points, key=lambda p: p.frame)
60     N = (pts[-1].frame + 1) if pts else 1
61     vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
62     for p in pts:
63         if 0 <= p.frame < N and p.visible and p.x >= 0:
64             vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)
65
66     mask = vis > 0
67     play_is_x = _spread(x[mask]) >= _spread(y[mask])
68     coord = x if play_is_x else y
69     net = float(np.median(coord[mask])) if mask.any() else 0.0
70     deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)
71
72     speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
73     max_gap = max(2, round(0.25 * fps)) # frames; a longer gap = occlusion break, not
motion
74     prev = None
75     for p in pts:
76         f = p.frame
77         if not (0 <= f < N and vis[f]):
78             continue
79         if prev is not None:
80             df = f - prev
81             if 0 < df <= max_gap:
82                 speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
83                 spd_mask[f] = 1.0
84                 c, pc = coord[f] - net, coord[prev] - net
85                 if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
86                     cross[f] = 1.0
87             prev = f
88     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
89
90
91 def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
92     """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
93     N = arr["N"]; fw = arr["fw"]
94     size = max(3, int(win_sec * fps) * 2 + 1)
95     vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
96
97     n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)

```

```

98     sm = uniform_filter1d(spd_mask, size, mode="constant")
99     spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
100    spd_max = maximum_filter1d(speed, size, mode="constant")
101    cross_rate = uniform_filter1d(cross, size, mode="constant") * fps          # crossings
/ second
102    xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
103    x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
104    x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
105    ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
106    y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
107    y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
108
109    idx = np.arange(0, N, max(1, int(stride_sec * fps)))
110    X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
111    return np.nan_to_num(X), idx / fps
112
113
114    def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
115        y = np.zeros(len(times))
116        for i, t in enumerate(times):
117            for s, e in gts:
118                if s <= t <= e:
119                    y[i] = 1.0
120                    break
121        return y
122
123
124    def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
125        """Rank-based AUC (Mann-Whitney) with MIDRANK tie handling.
126
127        Uses scipy.stats.rankdata (average method) so tied probabilities get averaged ranks
128        ?
129        without this, an all-tied classifier scored a spurious order-dependent 0.25..0.75
130        instead of 0.5, and dead-time frames produce identical all-zero feature rows
131        (guaranteed
132        ties), so the headline AUC carried an unquantified bias."""
133        pos, neg = p[y > 0.5], p[y <= 0.5]
134        if pos.size == 0 or neg.size == 0:
135            return float("nan")
136        ranks = rankdata(p) # midranks (ties averaged)
137        return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))
138
139
140    def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
141        merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
142        p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
143        on = p >= threshold
144        runs: List[Interval] = []
145        i, n = 0, len(on)
146        while i < n:
147            if on[i]:
148                j = i
149                while j + 1 < n and on[j + 1]:
150                    j += 1
151                runs.append((float(times[i]), float(times[j])))
152                i = j + 1
153            else:
154                i += 1
155        merged: List[Interval] = []
156        for s, e in runs:
157            if merged and s - merged[-1][1] <= merge_gap:
158                merged[-1] = (merged[-1][0], e)

```

```

157         else:
158             merged.append((s, e))
159     return [(s, e) for s, e in merged if e - s >= min_dur]
160
161
162 def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
163     f1s = []
164     for v in vids:
165         segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
166         f1s.append(score(segs, v["gts"], 0.5).f1)
167     return sum(f1s) / max(len(f1s), 1)
168
169
170 def run(specs: List[Dict]) -> dict:
171     data = {}
172     for s in specs:
173         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
174         arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
175         X, times = features(arr, float(s["fps"]))
176         gts = load_golden_gts(s["labels"])
177         data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
178     names = list(data)
179     print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
180     print(f"features: {FEAT_NAMES}\n")
181
182     results = []
183     for test in names:
184         train = [data[n] for n in names if n != test]
185         Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
186         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
187         clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
188         # tune the decision threshold on TRAIN segment-F1 (no test leakage)
189         thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
190         v = data[test]
191         probs = clf.predict_proba(v["X"])
192         auc = roc_auc(v["y"], probs)
193         segs = probs_to_segments(probs, v["times"], float(thr))
194         sc = score(segs, v["gts"], 0.5)
195         # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
196         # from the threshold-transfer gap. Optimistic ? for diagnosis only.
197         othr = max(np.arange(0.2, 0.81, 0.05),
198                   key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
199         osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
200         results.append({"video": test, "thr": float(thr), "auc": auc,
201                       "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
202                       "n_pred": len(segs), "n_gt": len(v["gts"]),
203                       "oracle_thr": float(othr), "oracle_f1": osc.f1})
204         print(f"[held-out {test}] per-frame AUC {auc:.3f} | LOVO-threshold F1
{sc.f1:.3f} "
205               f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train) | "
206               f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
207     mean_f1 = sum(r["f1"] for r in results) / len(results)
208     print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
209     return {"per_video": results, "mean_f1": mean_f1}
210
211
212 def main() -> None:
213     ap = argparse.ArgumentParser(description=__doc__)

```

```

214         ap.add_argument("--manifest", required=True, help="JSON:
[{'name','trajectory','fps','frame_width','labels'}, ...]")
215         args = ap.parse_args()
216         with open(args.manifest, encoding="utf-8") as f:
217             run(json.load(f))
218
219
220     if __name__ == "__main__":
221         main()
222

```

3. user (2026-06-10T09:28:19.114Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7     The ABC lets us:
8     - Keep the current WSL adapters working unchanged for Windows users.
9     - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33        TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34        deliberately static and model-agnostic). If the new detector needs different
35        windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37        registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39        DetectorRunner instances (they already do the right thing via duck-typing + the
40        type hints we add here).
41     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42        returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43     7. No change to eval/calibration code paths (they consume the CSV + the static
44        windowing helpers directly).
45
46     Optional Linux-native path (the main de-risk goal):
47     A LinuxNativeRunner (or ONNXRunner) would:
48     - Skip all wsl / _stage / conda activate.
49     - Load weights locally (torch or onnx) or call a native binary.
50     - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51     - healthcheck can just check "weights file exists and torch importable + CUDA?".
52     This removes the WSL requirement for Linux users / future SaaS workers while

```

```

53     keeping the exact same candidate quality characteristics (or better, once native
54     weights are available).
55
56     Error surfacing into reports:
57     The healthcheck message is already logged by the hybrids on failure. The reporting
58     harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59     segmenter Results (or pass detector_health into emit_indexer_report context), the
60     (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61     stage. The ABC makes that future change local to one place.
62
63     See also:
64     - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65     - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66     - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67     """
68
69     from abc import ABC, abstractmethod
70     from typing import Optional, Tuple
71
72
73     class DetectorRunner(ABC):
74         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
75
76         @abstractmethod
77         def run_predict(self, video_win_path: str, output_win_dir: str,
78                        video_id: Optional[str] = None) -> Optional[str]:
79             """Run inference and return path to trajectory CSV (or None on failure).
80
81             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
82             videos that happen to share a filename cannot reuse each other's frames or CSV.
83             It is optional for back-compat; callers in the live pipeline always pass it.
84             """
85             pass
86
87         @abstractmethod
88         def healthcheck(self) -> Tuple[bool, str]:
89             """Quick check if the runner (env, weights, etc.) is usable.
90
91             Returns (ok, message). Never raises. Cheap to call before a long run.
92             """
93             pass
94

```

4. user (2026-06-10T09:28:28.267Z)

```

1     # Owned-model implementation plan (roadmap) ? DRAFT for owner review
2
3     **Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner
4     greenlights a specific
5     milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
6     training source).
7
8     **Grounded in:** `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO +
9     learned prototype
10    (`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy
11    workflow (2026-06-09).
12
13    ## Design principle (owner directive): *envision now, build later*
14    Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be
15    **architecturally envisioned
16    now** so we never hit a mammoth refactor. We **design the seams** (data contracts,
17    export boundaries, the
18    event store) up front and **implement behind them** when each milestone is greenlit.
19    Punting a *feature* is fine;
20    punting its *seam* is not.
21

```

```

14     ## North Star (reframed per PR #61 review)
15     - **Immediate deliverable:** a **training framework + harness** that produces a model
with **very high precision/
16     recall for extracting highlights + rallies from a clip**. *Quality is the first-order
objective.*
17     - **Eventual product:** a **single, sport-agnostic, conversational** video model ?
upload a clip, then ask
18     contextual questions ("longest rally", "make a clip of all service faults", "rallies
over 20 shots").
19     - **On-device is LATER** (a model-compression goal so power users save upload
cost/hassle) ? definitely on the
20     roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
21     - **The moat = the consented dataset + the eval harness, not the weights.**
22
23     ## Hard guardrails (non-negotiable ? "never corrupted")
24     - **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT
/ Apache-2.0 / BSD only.**
25     (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-
restrictive: Gemma 3 (viral),
26     **Gemma 3n** (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU
cap), Commons-Clause repos.
27     - **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public
grounding-CoT datasets that
28     were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision
(a real, binding cost).
29     - **Exclude minors; per-user training data needs a separate explicit opt-in; split data
by match.**
30     - ? **Base-model pretraining provenance is unauditale for *every* open VLM (incl.
Qwen3-VL).** Our "no paid-LLM
31     training" rule is enforceable at *our fine-tuning data* layer, not the base's
pretraining ? an **explicit owner
32     risk-acceptance** is required to use any open VLM. (Governed by the base's license,
not our pipeline.)
33
34     ## The model strategy (verified 2026-06-09)
35     **Framework ? base** (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge
(runs SFT/GRPO, emits
36     weights); a *base* is the metal you forge. You pick one framework and feed it any clean
base.
37     - **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio
+ timestamp-grounding +
38     GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core;
*avoid its AGPL Studio UI*)
39     as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-
time snapshot ? re-verify at impl.
40     - **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-
video context, purpose-built
41     **timestamp emission** (second-level localization), multi-turn video chat. Re-confirm
the exact size's LICENSE
42     pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but
its Prohibited-Use posture is
43     unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not
the primary extractor.
44     **InternVL3 (MIT/Apache)** = clean fallback if a Qwen blocker appears. ? base swap is
cheap *only within
45     timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-
specific).
46     - **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss
strict temporal boundaries**
47     (best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse
regions, not accurate boundaries").
48     **Boundary accuracy IS the product.** The head (`rally_seq_proto.py`) is already de-
risked on our corpus (held-out
49     AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params
(trivially compressible later),

```



```

50         and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So:
**head now ? VLM later**,
51         strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes,
head refines boundaries) over
52         wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur
footage may differ ? re-measure.)*
53
54         ## Conversational video-QA ? build the bookkeeping NOW, punt the model
55         **Punt the conversational *feature*** until extraction quality is solid (current LOVO
~0.583 ? "very high"; a chat UI
56         over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn**
(expensive; bad at exact
57         counting; the strong teachers are license-forbidden). **But build the *seam* now** (the
brief requires it):
58         - **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits
structured timestamped
59         spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-
SQL + ffmpeg clip-cut). The
60         cited queries ("longest rally", "all service faults", "rallies > 20 shots") are
**structured-numeric ? SQL, not a
61         VLM job** (SQL counts exactly where VLMs hallucinate).
62         - **Designed-now seam:** make the **per-video event-index schema a first-class data
contract** alongside the M0.1
63         corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count,
ending_reason/fault_tags,
64         derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy
(e.g. `service_fault`) up front**
65         even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration):
`play_segments` today has
66         `video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights`
isn't a table yet ? so it's an
67         *extension with new columns + a highlights table*, not a no-op. **Punt** the NL
parser/agent until quality clears the bar.
68         - Paid Gemini may answer genuinely open-ended edge questions **at inference** over our
structured index (allowed ?
69         it reasons over our data, it is not trained on its outputs).
70
71         ## Multisport (single sport-agnostic model)
72         - **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned
across sports (sport = an input
73         feature / per-sport calibration). The collector + indexer are already sport-generic. ?
**ACTION: rename the repo off
74         "badminton" ASAP** (owner emphatic) ? consistent with this.
75         - **What multisport changes = DATA + DETECTORS (scale up, the accepted "good
problem"): ** each new sport needs its own
76         labeled matches (split by match) + a **clean trajectory detector** to feed the head's
features. **The binding blocker
77         is per-sport detectors: ** WASB (badminton) has the unresolved **C3** checkpoint-
provenance issue, and **clean MIT/
78         Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet
identified. ** Sequence by public-data
79         richness: **badminton ? tennis/TT ? pickleball/padel.**
80         - ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not
proven ? our corpus is badminton-only,
81         and the evidence once cited for it (RacketVision) was *refuted on verification* (it's
ball-tracking, not temporal). The
82         concept is sound on first principles (rally signal = a racquet-sport invariant) but
must be validated once a 2nd-sport
83         corpus exists. Don't cite RacketVision as evidence.
84
85         ## Language ? Python everywhere; on-device runtime is a deferred, layer-local export
shell
86         - **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer);
offline/batch, no latency hot path.
87         - **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-

```

```

process (ffmpeg, WASB-in-WSL, cloud
88     GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ?
single-digit-ms wins dwarfed by
89     ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval
harness + provider registry.
90     - **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate
our dev language:** train in
91     Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device
shell (Swift/Kotlin/C++/Rust)
92     runs the artifact with zero Python. **The one concrete constraint now: train with
export-clean ops** so the compression
93     target stays reachable without re-architecting. (On-device base stays on the
Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
94     - Escalation order if a real Python CPU hot path ever appears: free-threading ?
numpy/vectorize ? Cython/PyO3 ? only then a
95     separate service. **None exists today.**
96
97     ---
98
99     ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) .
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg . TAL
ActivityNet-JSON . VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus . **DONE (n=6), running** . *owner action + my scoring loop*
6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
110     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
111     data + calibration, not method.** Keep growing per new sport.
112
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) .
*greenlight item*
115     - **(i) PURGE the Gemini-silver TRAINING path** ? the live no-paid-LLM-training
violation is
116     **`backend/eval/distill_local.py`** (it trains a classifier on Gemini-silver CSV
labels ? `output/local_rally_model.json`)
117     and the threshold distillation in `calibrate_local.py`. (NB:
`backend/eval/training.py::label_candidates` is a *pure,
118     source-agnostic* function ? there is no store by that name; the earlier
"training.label_candidates" pointer was wrong.)
119     Nothing Gemini-trained is in git (artifacts live under gitignored `output/`), but the
path that regenerates them is one
120     command away ? retarget to human + open-teacher (own-model/WASB) labels; Gemini =
eval/reference/fallback only.
121     - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
122     blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean MIT/Apache detector swap.
123     ? **own-weights is an un-scoped NEW workstream, not ROADMAP Phase 4:** per ADR-002 our
temporal labels can't train the
124     shuttle detector (it needs per-frame coordinate labels we don't have). **Multiplies
per sport** (incl. any TT-on-WASB use).

```

```

125     Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
126
127     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) . *greenlight item*
128     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
129     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
130     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
131     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
132     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
133
134     ---
135
136     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
137     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
138     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
overfitting to declare victory.
139
140     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
141     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
142     (PR #35) found it weak (~2.2x discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
143     parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
144     - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
145     our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
146     - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) *** a real
147     VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
148     largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
149
150     ---
151
152     ## Decisions / status
153     | # | item | status |
154     |---|---|---|
155     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
156     | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
157     | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
158     | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
159     | 5 | Rename repo off "badminton" | **owner action** ? |
160     | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
161
162     ## Open risks / unknowns (none block Phase-0 research)
163     Budget/cost to reach Gen-2 **unquantified** (largest risk) . ship-gate "very high P/R"
target undefined . per-sport clean
164     trajectory detectors not yet identified . base-model pretraining provenance unauditale
(risk-acceptance) . real VLM
165     VRAM/token cost . single-model-multisport-temporal generalization unproven . DPA + no-
train (minors-excluded) for cloud
166     training . on-device 4B latency on 8 GB unmeasured.
167
168     ---

```

```
169      *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
170      next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
171
```

5. user (2026-06-10T09:28:29.306Z)

```
Exit code 2
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/:
__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py
```

6. user (2026-06-10T09:28:36.724Z)

No files found

7. user (2026-06-10T09:28:37.128Z)

```
1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth  ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8      predictions   ? the production pipeline's stored DB rows    [backend.eval.harness]
9      comparison    ? interval matching + P/R/F1                  [backend.eval.metrics]
10     baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
```

```

30
31 # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32 # actually load a committed baseline to compare against.
33 DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34 DEFAULT_TOLERANCE = 0.05
35
36
37 def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38     """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39     norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40     return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43 @dataclass
44 class RegressionResult:
45     video_id: str
46     stage: str
47     iou_threshold: float
48     precision: float
49     recall: float
50     f1: float
51     # Baseline values compared against (None when no baseline existed yet).
52     baseline_precision: Optional[float]
53     baseline_recall: Optional[float]
54     tolerance: float
55     regressed: bool
56     reasons: List[str]
57     # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58     # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59     has_baseline: bool = False
60     gt_hash: Optional[str] = None
61
62     def as_dict(self) -> dict:
63         return asdict(self)
64
65
66 # ----- baseline store
67
68 def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69     """Load the per-video baseline snapshot file (returns {} if absent)."""
70     if not os.path.isfile(path):
71         return {}
72     with open(path) as f:
73         return json.load(f)
74
75
76 def _baseline_key(video_id: str, stage: str) -> str:
77     return f"{video_id}::{stage}"
78
79
80 def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                  f1: float, path: str = DEFAULT_BASELINE_PATH,
82                  iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                  gt_fingerprint: Optional[str] = None) -> None:
84     """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86     Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87     later run computed under different settings (or against different labels) is
detected
88     as incommensurable rather than silently compared.
89     """
90     baselines = load_baselines(path)
91     baselines[_baseline_key(video_id, stage)] = {

```

```

92         "video_id": video_id, "stage": stage,
93         "precision": precision, "recall": recall, "f1": f1,
94         "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95     }
96     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97     with open(path, "w") as f:
98         json.dump(baselines, f, indent=2, sort_keys=True)
99     logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105           stage: str = "final", iou_threshold: float = 0.5,
106           detector: Optional[str] = None,
107           tolerance: float = DEFAULT_TOLERANCE,
108           baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -

```

```

{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
170         regress.
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
188             tol={r.tolerance}"
189         return head + " ".join(f"\n    - {why}" for why in r.reasons)

```

8. user (2026-06-10T09:28:46.898Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
17         # module-level import error shipping while the suite stays green.

```

```

18     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19     # on machines without the GPU stack (test_import_smoke stubs what's absent).
20     import-smoke:
21         name: Syntax sweep + import smoke (no GPU stack)
22         runs-on: ubuntu-latest
23         steps:
24             - uses: actions/checkout@v4
25             - uses: actions/setup-python@v5
26               with:
27                 python-version: "3.13"
28             - name: Install minimal deps
29               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
30             - name: Run import smoke tests
31               run: python -m pytest tests/test_import_smoke.py -v
32
33     full-suite:
34         name: Full test suite (offline, mocked)
35         runs-on: ubuntu-latest
36         steps:
37             - uses: actions/checkout@v4
38             - uses: actions/setup-python@v5
39               with:
40                 python-version: "3.13"
41                 cache: pip
42             - name: System libs for opencv-python
43               run: |
44                 sudo apt-get update
45                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
46             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48             - name: Install requirements
49               run: python -m pip install -r requirements.txt pytest httpx # httpx: fastapi
TestClient
50         # obsreport (private, soft dep) is absent here: reporting tests skip via
51         # pytest.importorskip ? run_all_tests.py prints which.
52         - name: Run full suite
53           run: python run_all_tests.py
54

```

9. user (2026-06-10T09:28:48.176Z)

```

34: def load_golden_gts(csv_path: str) -> List[Interval]:
35-     """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36-     gts: List[Interval] = []
37-     with open(csv_path, newline="", encoding="utf-8") as f:
38-         reader = csv.DictReader(f)
39-         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or [])]
40-         for row in reader:
41-             row = {(k or "").strip().lower(): v for k, v in row.items()}
42-             try:
43-                 s = float((row.get("start_time") or "").strip())
44-                 e = float((row.get("end_time") or "").strip())
45-             except (TypeError, ValueError):
46-                 continue
47-             if e > s:
48-                 gts.append((s, e))
49-     return sorted(gts)
50-
51-
52: def make_predict_fn(trajjectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53-     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""

```



```

54-     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
55-
56-     def predict_fn(cfg: Config) -> List[Interval]:
57-         vt, mg, mwd = cfg
58-         wins = TrackNetRunner.trajectory_to_action_windows(
59-             points, fps=fps, frame_width=frame_width,

```

10. user (2026-06-10T09:28:58.356Z)

```

1:from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2:from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3:from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4:from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5:from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6:from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

```

11. user (2026-06-10T09:28:59.690Z)

```

C:\Users\avidu\Projects\sports-data-collector\README.md:16:- **Interactive Curation CLI & Visual
Play Harness**: Let's you query DB status, list staging entries, review sample timestamps, and
approve/reject them. It includes a built-in play function that opens the video in your default
browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files
or YouTube start time markers `&t=start` for remote videos) for manual visual verification.
C:\Users\avidu\Projects\sports-data-collector\README.md:97:To turn curated annotations into
ground-truth files the badminton-highlight-indexer can score and train against, export per-video
`start,end` CSVs plus a `manifest.json`:
C:\Users\avidu\Projects\sports-data-collector\README.md:105:[Omitted long matching line]
C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md:28: `start,end` CSVs + a
`manifest.json` pairing each CSV with its video file.
C:\Users\avidu\Projects\sports-data-
collector\docs\MANUAL_INGESTION_POSITIONING.md:17:`manifest.json` + a per-video `start,end` CSV
(`curate.py::export_eval_set`;
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:32: (rich
14-col CSV) everything but (start,end) is manifest.json +
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:33: ? vendor
Roora / CVAT validated then dropped before per-video start,end CSV
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:39:The lean
annotator CSV header `rally_number,start_time,end_time,ending_reason,sport`
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:49:python -m
src.cli.curate export # ? manifest.json + <video_id>.csv (start,end)
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:119: a
round-trip test (header matching is exact case-insensitive, so a vendor
C:\Users\avidu\Projects\sports-data-collector\data\roora_badminton_canonical.csv:1:rally_number,
start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot_outcome,score_before
,score_after,notes
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md:18:
? SQLite (rally ? ?????????? ? <id>.csv ? start,end (indexer GT)
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md:42:# 4. Export indexer-
ready ground truth (start,end CSVs + manifest)
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:23:
rally_number,start_time,end_time,ending_reason,sport[,shots_count]
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:195:
reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or [])]
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:200: w =
csv.DictWriter(f, fieldnames=OUT_FIELDS)
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:201:
w.writeheader()
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:49: print(tabulate(data,
headers=["Sport", "Status", "Count"], tablefmt="grid"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:295: # Normalize
headers
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:296:
reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if reader.fieldnames else []
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:489:
headers=["Video ID", "Action", "Detail"], tablefmt="grid"))

```

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:552:         indexer's
`start,end` ground-truth format (decimal seconds), plus a single
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:637:
print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?", "Video file"],
tablefmt="grid"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:796:
print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport", "License",
"Commercial Safe?"], tablefmt="grid"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:845:
print(tabulate(sample_data, headers=["Rally #", "Duration", "Score", "Shots"],
tablefmt="simple"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:851:
print(tabulate(sample_data, headers=["Point #", "Duration", "Score"], tablefmt="simple"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:857:
print(tabulate(sample_data, headers=["Over.Ball", "Duration", "Runs", "Wicket"],
tablefmt="simple"))
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:945:         # Create file://
URL with media fragment (browsers support #t=start,end fragment)
C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py:198:         "rally_numbe
r,start_time,end_time,start_mmss,end_mmss,shots_count,ending_reason,last_shot_outcome\n"
C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py:213:
"rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"

[Showing results with pagination = limit: 30]

```

12. user (2026-06-10T09:29:23.638Z)

```

backend\pipeline\detectors\wasb_runner.py:34:     repo_dir: str = "~/models/WASB-SBDT"
backend\pipeline\detectors\wasb_runner.py:37:     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend\pipeline\detectors\wasb_runner.py:47:         repo_dir=w.get("repo_dir",
"~/models/WASB-SBDT"),
backend\pipeline\detectors\wasb_runner.py:51:         "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
backend\pipeline\detectors\wasb_infer.py:3:WASB-SBDT only ships *dataset* evaluation; this is a
thin wrapper that reuses its
backend\pipeline\detectors\wasb_infer.py:10:It must run INSIDE the WSL `wasb` conda env, from
the WASB-SBDT `src/` dir (it
backend\pipeline\detectors\wasb_infer.py:35:Usage (in WSL, from ~/models/WASB-SBDT/src):
backend\pipeline\detectors\base.py:42:     returns str|None) using mocks for the external bits ?
see test_tracknet_runner.py.

```

13. user (2026-06-10T09:29:23.660Z)

```

7:> **2026-06-10 sweep (12 PRs, both repos, all merged):** CI now exists in both repos; master
un-broken (compiler
16:Build the **quality-first extraction harness** before anything else: a learned **TAL head**
(ActionFormer/ASFormer, MIT)
37:   - **M0.4 ? Gen-0 TAL harness** (first-class): productionize `rally_seq_proto.py` ? one-
command train?eval?**ship-gate** over
41:   - **M0.1 ? corpus spine + the per-video event-index contract** (the conversational seam:
extract ? event store ? text-to-SQL).
57:   - **Gemini-wrapper baseline vs the n=6 LOVO corpus** (harness exists) ? if a wrapper ?
learned 0.626, the owned-model
59:4. **Housekeeping:** **rename repo off "badminton"** (multisport, owner action ? now in
BACKLOG); confirm collector =

```

14. user (2026-06-10T09:30:44.976Z)

```

CLAUDE.md
README.md
TEST_RUN_SUMMARY.md
__pycache__
backend

```

```
config.json
docs
output
pytest.ini
requirements.txt
run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
scratch
scripts
setup.py
tests
docs/archives/checkpoints/2026-06-eval-baseline/README.md
```

15. user (2026-06-10T09:30:46.331Z)

```
107: def save(self, path: str) -> None:
108-     with open(path, "w") as f:
109-         json.dump(self.to_dict(), f, indent=2)
110-
111-
112: def load(cls, path: str) -> "LogisticRegression":
113-     with open(path) as f:
114-         return cls.from_dict(json.load(f))
```

16. user (2026-06-10T09:30:54.142Z)

```
60         else:
61             w_pos = w_neg = 1.0
62             sw = np.where(y == 1, w_pos, w_neg)
63
64             self.w = np.zeros(d)
65             self.b = 0.0
66             for _ in range(self.epochs):
67                 p = self._sigmoid(Xs @ self.w + self.b)
68                 err = (p - y) * sw
69                 grad_w = Xs.T @ err / n + self.l2 * self.w
70                 grad_b = float(err.mean())
71                 self.w -= self.lr * grad_w
72                 self.b -= self.lr * grad_b
73             return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
```

```

96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))

```

17. user (2026-06-10T09:32:49.615Z)

Structured output provided successfully

18. assistant (2026-06-10T09:32:55.248Z)

Position delivered via structured output.